NETWORK INTRUSION DETECTION (NIDS): PROJECT GUIDE & VIVA MANUAL

Enhanced Network Intrusion Detection Using Hybrid Autoencoder and Feature-Selective Machine Learning on CIC-IDS2017 Traffic Data

Research Paper Title:	Enhanced Network Intrusion Detection Using Hybrid Autoencoder and Feature-Selective Machine Learning on CIC-IDS2017 Traffic Data
Authors & Affiliation:	Saikat Mondal, Surya Pritam Satpathy, Sachin Kailas Borse | School of Computer Engineering, KIIT Deemed to be University
Dataset Evaluated:	CIC-IDS2017 Benchmark (Canadian Institute for Cybersecurity, 8 Attack Vectors)
Key Architecture:	Deep Hybrid Autoencoder (Unsupervised Latent Extraction) + Feature Selection + Ensemble Random Forest
Target Publication:	IEEE International Conference on Research in Computational Intelligence and Communication Networks (ICRCICN 2027)

1. The Big Picture: Explained Simply for Anyone

Imagine you are explaining this project to a first-year student or to your project guide who wants to understand the exact motivation without getting lost in mathematical jargon:

`[THE AIRPORT SECURITY & CHEAT-SHEET ANALOGY (REMEMBER THIS FOR VIVA)]` Think of a busy international airport. Every minute, thousands of passengers walk through the security gate. A traditional security guard (Signature-Based NIDS) has a book of photos of known wanted criminals. If a criminal changes their hairstyle, wears sunglasses, or is a brand-new criminal (zero-day exploit), the guard lets them pass! Now, you decide to use Machine Learning to watch for 'suspicious behavior' (Anomaly Detection). But for every passenger, you are recording 80 different measurements: height, shoe size, heartbeat, walking speed, luggage weight, etc. Your system gets completely overwhelmed by 80 columns of noisy data—it lags, slows down, and starts sounding false alarms on innocent passengers. This project solves that exact problem using a two-step brain filter: First, an Autoencoder compresses those 80 noisy measurements into a compact cheat-sheet of core patterns. Second, a Feature Selection filter removes any remaining useless variables, leaving only the sharpest attack signals. Finally, an ensemble classifier makes the decision with 99.93% accuracy and almost zero false alarms (0.10% FAR).

1.1 Signature-Based vs Anomaly-Based NIDS

- Signature-Based Detection (e.g., Snort, Suricata): Compares incoming packets against a static database of known attack signatures. Very fast, but completely blind to zero-day exploits, mutated malware, and encrypted payloads.

- Anomaly-Based Detection (Machine Learning): Establishes a mathematical baseline of 'normal' traffic and flags any statistically significant deviation as an intrusion. Capable of catching zero-day attacks, but historically plagued by high False Positive Rates (FPR).

1.2 The 'Curse of Dimensionality' in Network Traffic

The Problem: Modern network flow extractors (like CICFlowMeter) generate over 80 statistical features per packet flow (inter-arrival times, packet length variance, flow duration, flag counts). Feeding 80 raw features into shallow ML models (SVM, Decision Trees) causes three fatal problems:

- Computational Latency: In a 10 Gbps network, the ML model cannot keep up with wire speed.
- Multicollinearity & Noise: Many features are redundant (e.g., total bytes forward vs total packets forward) or contain random noise, which misleads the classifier.
- Overfitting: The model memorizes training noise instead of learning genuine attack patterns, failing when tested on unseen network flows.

1.3 What is an Autoencoder (AE)?

An Autoencoder is a specialized neural network designed for unsupervised representation learning. Think of it as an expert book summarizer:

- The Encoder: Compresses an 80-page document into a compact 16-page 'summary' (the bottleneck latent space z).
- The Decoder: Takes that 16-page summary and tries to reconstruct the original 80-page document.
- The Loss Function: Measures how much was lost in reconstruction (Mean Squared Error). If the decoder can accurately reconstruct the original data from the 16-page summary, it proves that the 16 latent variables captured the fundamental underlying structure of the network traffic while throwing away stochastic noise!

1.4 Why a 'Hybrid' Approach? (Autoencoder + Feature Selection)

Most researchers do one of two things: they either do Feature Selection on raw data (which misses complex non-linear relationships), OR they feed raw Autoencoder latent features directly into a classifier (which still retains non-discriminative latent noise).

Our Key Innovation: Our innovation is a unified sequential hybrid pipeline: We first use the Autoencoder to capture non-linear structural manifolds, and THEN apply Feature Selection on the latent space to prune any uninformative latent dimensions. This double-filtering achieves maximum signal-to-noise ratio.

2. The Core Problem This Project Solves

When your guide asks: 'Why did you build this? What is the practical value of your research?', explain these three key operational challenges:

Problem 1: SOC Alert Fatigue & False Alarm Rate (FAR): In enterprise Security Operations Centers (SOCs), analysts receive thousands of intrusion alerts every day. If 95% of them are false alarms, analysts suffer from 'alert fatigue' and start ignoring warnings. A single overlooked true positive can result in a catastrophic ransomware breach. By reducing the False Alarm Rate (FAR) to just 0.10%, our model restores analyst trust.

Problem 2: Non-Linear Traffic Dynamics: Standard Principal Component Analysis (PCA) performs linear dimensionality reduction. However, modern network protocols (HTTP/2, TLS handshakes, packet inter-arrival distributions) exhibit highly non-linear, chaotic dynamics. The deep autoencoder successfully captures non-linear manifolds that PCA completely misses.

Problem 3: Real-Time Edge Viability: While end-to-end deep neural networks (like Deep LSTMs or CNNs) can detect intrusions, they are computationally heavy and require GPU clusters for inference. Our hybrid architecture uses the deep autoencoder for offline latent extraction, and a lightweight Random Forest ensemble for online classification, enabling low-latency deployment at the network edge.

3. The Technical Pipeline: How It Works Step-by-Step

Here is the exact algorithmic workflow of the proposed system:

Step 1: Dataset Acquisition & Data Hygiene (CIC-IDS2017)

- Dataset Choice: We selected the CIC-IDS2017 benchmark generated by the Canadian Institute for Cybersecurity. It covers 5 days of realistic background traffic and 8 contemporary attack classes: DoS/DDoS (Slowloris, Hulk, GoldenEye), PortScan, Brute Force (SSH, FTP), Web Attacks (SQLi, XSS), Infiltration, and Botnets.
- Handling Missing & Infinite Values: Network flow statistics often produce divide-by-zero errors (e.g., flow bytes per second when duration is zero). We purged all `NaN` and `Inf` values to ensure mathematical stability.
- Z-Score Normalization (StandardScaler): Features like flow duration (ranging in millions of microseconds) would dominate features like flag counts (0 or 1). We normalized every continuous variable to have zero mean (mu = 0) and unit variance (sigma = 1): z = (x - mu) / sigma.
- Train/Test Partition: Evaluated using an 80/20 train-test partition with random shuffling to eliminate temporal sequence bias.

Step 2: Unsupervised Representation Learning (Deep Autoencoder)

- Encoder Architecture: Maps high-dimensional input vector x in R^D to compressed bottleneck latent code z in R^d (where d << D).
- Decoder Architecture: Reconstructs the original vector x_hat in R^D from the latent code z.
- Loss Minimization: Trained using Adam optimizer to minimize Mean Squared Error (MSE) reconstruction loss: L(x, x_hat) = (1/n) * sum_{i=1}^n (x_i - x_hat_i)^2.
- Result: The encoder discards high-frequency stochastic noise while preserving the underlying topological manifold of normal and malicious traffic.

Step 3: Strategic Feature Selection on Latent Space

- Latent Filtering: Unlike conventional systems that perform feature selection on raw data, we evaluate the predictive utility of the learned latent embeddings.
- Information-Theoretic Metric: Using Mutual Information / Gini Importance, latent variables that have low variance or zero correlation with attack labels are pruned.
- Benefit: Strips away redundant latent dimensions, maximizing the classification margin and reducing inference latency.

Step 4: Supervised Ensemble Classification (Random Forest)

- Model: Random Forest multi-tree ensemble classifier operating on the refined latent feature manifold.
- Voting Mechanism: Aggregates decision trees using bootstrap aggregating (bagging) to stabilize decision boundaries and prevent overfitting.
- Output: Binary classification decision: Normal Traffic vs Malicious Intrusion.

4. What We Achieved: Empirical Results & Analysis

The performance of the hybrid framework was validated on the CIC-IDS2017 benchmark. Here are the exact numbers reported in the research paper:

Evaluation Metric	Experimental Value	Practical Operational Meaning
Accuracy	**99.93%**	Overall probability that a network flow is correctly categorized.
Precision	**99.93%**	When the model flags an intrusion, it is genuinely malicious 99.93% of the time.
Recall (Sensitivity)	**99.95%**	The system successfully detects 9,995 out of every 10,000 actual cyber attacks.
F1-Score	**99.94%**	Harmonic mean of precision and recall, proving an optimal balance between catching attacks and avoiding false alarms.
False Alarm Rate (FAR)	**0.10%**	Only 1 out of every 1,000 normal network flows is erroneously flagged, eliminating SOC alert fatigue.

Confusion Matrix Analysis (Figure 3 in Paper)

On the test partition of ~197,500 unseen network flows:

- True Negatives (Normal classified as Normal): 98,000 flows.
- True Positives (Attack classified as Attack): 99,500 flows.
- False Positives (Normal falsely flagged as Attack): ~100 flows (FAR = 0.10%).
- False Negatives (Attacks missed): ~45 flows (Miss rate = 0.05%).
- Total Misclassifications: Only 145 out of 197,500 test samples!

5. Technologies We Used & Why

- Python 3: The industry standard for scientific machine learning and data science.
- TensorFlow / Keras: Utilized to construct and train the deep autoencoder architecture with custom reconstruction loss functions and GPU acceleration.
- Scikit-Learn: Used for data scaling (`StandardScaler`), feature selection routines, train-test splitting, and Random Forest ensemble modeling.
- Pandas & NumPy: High-throughput memory-mapped data ingestion and manipulation of multi-gigabyte CSV telemetry files.

- Matplotlib / Seaborn: Generation of ROC curves, confusion matrices, and performance bar charts.

6. Honest Critique of Current Paper Draft & What Needs Polishing

To prepare for questions from your guide, you must be 100% transparent about the difference between the current draft paper and a final published IEEE manuscript:

[CURRENT STATUS: WORKING DRAFT] The manuscript you have right now is an INTERMEDIATE DRAFT. It contains valid experimental results (99.93% accuracy, 0.10% FAR), but the document formatting and structure need final polish before submission to IEEE ICRCICN 2027.

Points to Fix in the Research Paper (Guide Defense Talking Points):

- Template Conversion: The draft is currently 15 pages in single-column. It needs to be migrated to the official IEEE 2-column format (max 6 pages).
- Add Section 2 (Related Work): The draft accidentally jumps from Section 1.3 to Section 3. You should explain to your guide: 'We have compiled recent 2021-2025 literature on Deep NIDS which will be inserted as Section II: Related Work in the IEEE 2-column camera-ready version.'
- Ablation Study Table: Reviewers will want to see the performance of Random Forest alone vs Autoencoder alone vs the combined Hybrid model. We will include this ablation table in the final submission.
- Binary vs Multiclass Scoping: Explain that binary classification was chosen as Phase 1 (high-speed frontline triage at the network boundary). Phase 2 of our research will introduce multi-label classification to pinpoint specific attack families (DoS vs PortScan vs Web Attacks).

7. Comprehensive Guide & Reviewer Viva Q&A Guide

Here are 12 of the most probable questions your guide or external examiners will ask regarding this NIDS research:

Q1: Why use an Autoencoder instead of standard Principal Component Analysis (PCA)?

How to Answer Confidently: PCA is strictly a linear transformation that projects data onto orthogonal eigenvectors. However, network traffic telemetry exhibits highly non-linear, chaotic distributions due to varying packet sizes, protocol negotiations, and attack payload patterns. Autoencoders use non-linear activation functions (such as ReLU or Sigmoid) across multiple hidden layers, allowing them to capture complex non-linear manifolds that PCA completely misses.

Core One-Liner to Remember: *"PCA is linear; Autoencoders learn complex non-linear traffic representations."*

Q2: Why apply Feature Selection AFTER the Autoencoder instead of before?

How to Answer Confidently: If you apply feature selection to raw data, you evaluate features independently, which ignores non-linear multi-feature interactions. Conversely, an Autoencoder maps the entire feature space into a compressed latent manifold, capturing those non-linear interactions. However, the latent bottleneck can still retain non-informative variance. Applying feature selection on the latent space ensures that we only feed the most discriminative latent dimensions into the classifier.

Core One-Liner to Remember: *"AE captures non-linear interactions; FS then prunes remaining latent noise."*

Q3: Why did you choose the CIC-IDS2017 dataset instead of KDD-99 or NSL-KDD?

How to Answer Confidently: KDD-99 and NSL-KDD are over 25 years old and reflect outdated 1990s traffic patterns that are irrelevant to modern networks. CIC-IDS2017 is a recognized contemporary benchmark developed by the Canadian Institute for Cybersecurity. It incorporates modern application-layer protocols, realistic background benign traffic, and 8 diverse modern attack families (including DDoS, Slowloris, Heartbleed, and Web Attacks).

Core One-Liner to Remember: *"CIC-IDS2017 reflects modern 21st-century attack vectors; KDD-99 is obsolete."*

Q4: What loss function is used to train the Autoencoder, and why?

How to Answer Confidently: We used Mean Squared Error (MSE) reconstruction loss: L(x, x_hat) = (1/n) * sum (x_i - x_hat_i)^2. In an autoencoder, the objective is to reconstruct the input x as closely as possible at the output layer x_hat. Minimizing MSE forces the network to learn a compressed representation in the bottleneck layer that retains maximum information with minimum distortion.

Core One-Liner to Remember: *"MSE loss measures the reconstruction error between raw input and decoded output."*

Q5: Why is the False Alarm Rate (FAR) of 0.10% so important in cybersecurity?

How to Answer Confidently: In modern enterprise networks processing millions of flows per minute, a false alarm rate of even 2% would generate tens of thousands of false alerts daily. This causes severe 'alert fatigue', leading SOC analysts to miss real attacks. Our model achieves a FAR of 0.10%, meaning only 1 in 1,000 benign flows is flagged, making it viable for practical real-world operations.

Core One-Liner to Remember: *"A low FAR of 0.10% prevents alert fatigue and preserves analyst resources."*

Q6: Why did you perform Binary Classification instead of Multi-class Classification?

How to Answer Confidently: In high-speed production networks, the primary operational priority is real-time frontline triage: immediately separating malicious packets from benign traffic with minimum latency. Our binary model acts as this high-throughput filter. In the future scope of our research, flows flagged as malicious can be routed to a secondary, specialized multiclass model to categorize the exact attack family.

Core One-Liner to Remember: *"Binary classification provides high-speed frontline triage at wire speed."*

Q7: How did you handle the class imbalance in CIC-IDS2017?

How to Answer Confidently: In raw CIC-IDS2017, benign traffic accounts for over 80% of all flows. To prevent the classifier from developing a majority-class bias toward benign traffic, we extracted a balanced stratified subset of normal and attack samples (~197,500 test flows) for our experimental evaluation. In operational deployment, Random Forest class weights can be adjusted to penalize false negatives.

Core One-Liner to Remember: *"Stratified balanced sampling was used to prevent majority-class bias."*

Q8: Why did you choose Random Forest as the final classifier instead of a Deep Neural Network?

How to Answer Confidently: Deep neural networks require significant computational resources and GPU

acceleration for inference, introducing latency. Random Forest is an ensemble of decision trees that executes extremely fast inference on CPU, handles tabular numerical features exceptionally well, provides natural resilience against overfitting via bagging, and is suitable for edge-router deployment.

Core One-Liner to Remember: *"Random Forest provides microsecond CPU inference and resistance to overfitting."*

Q9: What preprocessing steps were applied to the raw CSV files?

How to Answer Confidently: First, we cleaned the data by purging instances containing missing (NaN) or infinite values caused by zero-duration flows. Second, we applied Z-score standardization (StandardScaler) to normalize continuous variables to zero mean and unit variance. Third, categorical labels were one-hot encoded, and data was randomly shuffled to prevent temporal sequence bias.

Core One-Liner to Remember: *"Cleaning NaN/Inf values, Z-score standard scaling, and random shuffling."*

Q10: How does this model prevent overfitting?

How to Answer Confidently: Overfitting is prevented at three distinct stages: (1) The bottleneck layer of the autoencoder acts as a structural regularizer by constraining information capacity; (2) Feature selection removes noisy, low-variance latent dimensions; and (3) Random Forest uses bootstrap aggregation (bagging) and feature subsetting across multiple decision trees to minimize model variance.

Core One-Liner to Remember: *"Three-stage regularization: AE bottleneck, feature pruning, and RF bagging."*

Q11: What are the limitations of this study?

How to Answer Confidently: Currently, the evaluation is based on offline flow-level CSV datasets rather than live raw PCAP packet streams. Second, the model performs binary categorization rather than multi-class attack attribution. Third, like all ML-based systems, model drift can occur over time as adversarial techniques evolve, requiring periodic retraining on updated traffic profiles.

Core One-Liner to Remember: *"Offline dataset evaluation, binary scope, and vulnerability to long-term model drift."*

Q12: What are your future research directions?

How to Answer Confidently: Our roadmap includes: (1) Extending the pipeline to multi-label classification to identify specific attack families; (2) Implementing online incremental learning to adapt to zero-day traffic shifts without retraining from scratch; and (3) Deploying the model onto edge hardware (such as Raspberry Pi or NVIDIA Jetson) to measure real-time throughput on a physical 10 Gbps switch.

Core One-Liner to Remember: *"Multi-class attack classification, incremental online learning, and edge hardware deployment."*